Assignment 3 | Bridge Pattern

School of Software Engineering
ShP-2216 Software Design Patterns | 2026-2027 | Yerassyl Bekenov
Topic A - Drawing

Identification pending: the student name, group and repository URL were not supplied. This report must be personalized before submission. The repository is currently local.

Two independently varying dimensions

The abstraction hierarchy contains Shape, Circle and Square. It owns each shape's ID and geometry: a radius of 2 for the sample Circle and a side of 3 for the sample Square. The implementation hierarchy contains Renderer and three rendering styles. Each style returns its own text description while preserving the supplied geometry.

Shape stores a Renderer interface reference supplied by the client. Circle.execute() selects renderCircle(radius); Square.execute() selects renderSquare(side). Neither abstraction knows which concrete renderer performs the operation. Composition makes the six combinations possible without six combination subclasses. This is the separation described by Bridge [1].

Runtime replacement: T5

```
Circle afterSwitch = original;
afterSwitch.setImplementation(new RasterRenderer());
String after = afterSwitch.execute();

boolean sameObject = original == afterSwitch;
boolean stateUnchanged = originalId.equals(afterSwitch.getId())
        && originalRadius == afterSwitch.getRadius();
```

The original reference, ID and radius are saved before this excerpt. The == expression proves identity; the getter comparisons prove preserved domain data. Only the private renderer reference changes. The demo also compares both real execute() results with independent expected strings.

Observed: sameObject=true; stateUnchanged=true; ID circle-switch -> circle-switch; radius 2 -> 2.
Before: VECTOR circle radius=2
After: RASTER circle radius=2

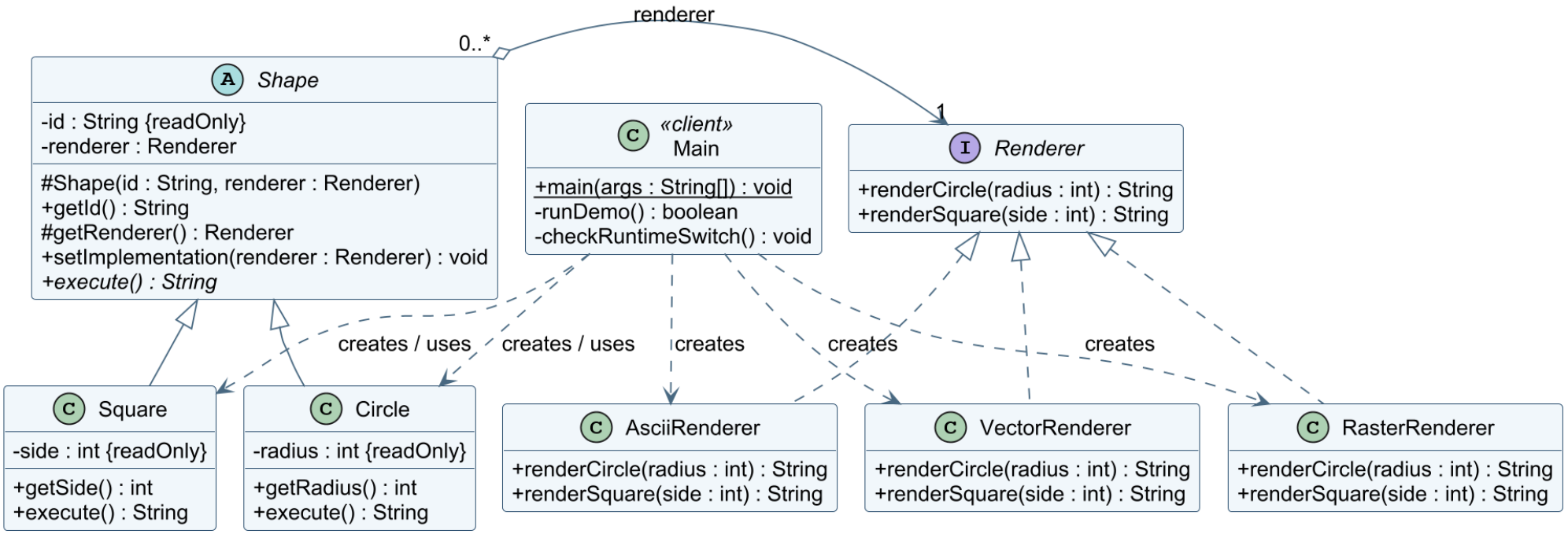
Reproduction and evidence

```
javac --release 17 -encoding UTF-8 -d out "@sources.txt"
java -cp out Main --demo
```

The captured program output in demo-output.txt runs T1-T7 in order and ends with **SUMMARY: 7/7 PASS**. T1-T4 cover both shapes with both initial renderers; T5 checks runtime replacement; T6-T7 cover the ASCII extension. Comparisons determine the labels, and counters determine the summary. Failures print the expected value and cause exit status 1.

Base commit: 423467d70f88646b802b9eb139b2db4efc1c2ec2
I3 source commit: be88bfaeeb7be8418ab0b2cd7ef11996e0e0555a
Packaged submission commit: recorded exactly in SUBMISSION.txt at the ZIP root. The packaged Java sources are unchanged from the I3 source commit.

UML class diagram



Reading the diagram: solid hollow triangles show inheritance; dashed hollow triangles show interface implementation. The open diamond and arrow represent Shape holding exactly one Renderer; a renderer may be shared by several shapes. Dotted arrows show the client's creation/use dependencies. '+' is public, '#' protected, '-' private. The diagram shows the methods relevant to the Bridge and demo; constructors of concrete classes and Main's output helpers are omitted.

Delegated call: Main constructs Circle with VectorRenderer -> Circle.execute() -> Shape.getRenderer() -> Renderer.renderCircle(2) -> VectorRenderer returns the result. Source: docs/bridge.puml, rendered with PlantUML.

Code quality and extension

1. Separate responsibilities

src/drawing/Circle.java:19-21

```
public String execute() {
    return getRenderer().renderCircle(radius);
}
```

Circle selects the shape operation and passes its geometry. The renderer owns the presentation, so style changes stay outside Circle.

2. Meaningful names

src/drawing/Renderer.java:3-6

```
public interface Renderer {
    String renderCircle(int radius);

    String renderSquare(int side);
```

renderCircle, radius, renderSquare and side identify the shape and the exact measurement at the contract boundary.

3. Small, focused methods

src/drawing/Shape.java:22-24

```
public final void setImplementation(Renderer renderer) {
    this.renderer = Objects.requireNonNull(renderer, "renderer");
}
```

setImplementation has one job: validate and replace the renderer reference. It does not reconstruct the shape or modify its data.

4. Avoid duplicated workflow

src/Main.java:72-74

```
private void checkResult(String testId, String classes, String actual, String expected) {
    record(testId, expected.equals(actual), classes + " | result=" + actual, expected);
}
```

One helper compares each ordinary result and forwards it to record(), which centralizes counters, labels and failure output.

5. Encapsulate state

src/drawing/Circle.java:3-4

```
public final class Circle extends Shape {
    private final int radius;
```

The private final radius is assigned in the constructor and exposed only by a getter. Shape uses the same approach for its ID.

Independent extension, trade-off and intent

The working I1/I2 version was committed first. The next commit added AsciiRenderer and updated Main for T6-T7; sources.txt and documentation were also updated. extension.diff proves that Shape, Circle, Square, Renderer, VectorRenderer and RasterRenderer stayed unchanged. Existing code calls the same Renderer contract.

The cost is more classes and indirection for a small simulation. Adding a new shape-specific primitive, such as renderTriangle, would also require changes to the interface and every renderer. Bridge separates planned dimensions so implementations can vary under a stable contract [1]. Adapter instead makes an incompatible existing API usable by translating calls [2]; no such legacy API is present here.

References and authorship

[1] Refactoring.Guru, *Bridge*. refactoring.guru/design-patterns/bridge
[2] Refactoring.Guru, *Adapter*. refactoring.guru/design-patterns/adapter
[3] Astana IT University, *Assignment 3 | Bridge Pattern*, ShP-2216, 2026-2027, user-provided brief. Web references consulted 4 October 2026.
Prepared with AI assistance. Review and adapt the work under the course rules and be able to explain every submitted method.